

EXPERIMENT 6

AIM: Develop a program that includes the features NESTED IF, CASE and CASE expression. The program can be extended using the NULLIF and COALESCE functions.

SOLUTION:

1. NESTED IF

A **Nested IF** is an IF statement written **inside another IF statement**.

It is used when a decision depends on another condition.

Syntax

```
IF condition1 THEN
    IF condition2 THEN
        statements;
    ELSE
        statements;
    END IF;
ELSE
    statements;
END IF;
```

2. CASE Statement

A **CASE statement** is a control structure used to execute different blocks of code based on conditions. It is similar to **IF–ELSE IF**.

Syntax

```
CASE
    WHEN condition1 THEN statements;
    WHEN condition2 THEN statements;
    ELSE statements;
END CASE;
```

3. CASE Expression

A **CASE expression** returns a **single value** and is used inside assignments or SQL statements.

Syntax

```
variable :=
CASE
    WHEN condition1 THEN value1
    WHEN condition2 THEN value2
    ELSE value3
END;
```

4. NULLIF Function

NULLIF(expr1, expr2) returns **NULL** if both expressions are equal, otherwise returns expr1.

Syntax

NULLIF(expression1, expression2)

NULLIF(marks, 0)

If marks = 0 → returns NULL

If marks = 50 → returns 50

5. COALESCE Function

COALESCE returns the **first non-NULL value** from a list.

Syntax

COALESCE(expr1, expr2, expr3, ...)

COALESCE(marks, 35)

If marks is NULL → returns 35

If marks is 60 → returns 60

Program:

```
SET SERVEROUTPUT ON;
```

```
DECLARE
```

```
    roll student.rollno%TYPE := 10;
```

```
    sname student.name%TYPE;
```

```
    smarks student.marks%TYPE;
```

```
    sgrade VARCHAR2(10);
```

```
    sresult VARCHAR2(20);
```

```
BEGIN
```

```
    -- Using COALESCE and NULLIF
```

```
    SELECT name,
```

```
           COALESCE(NULLIF(marks, 0), 0)
```

```
    INTO sname, smarks
```

```
    FROM student
```

```
    WHERE rollno = roll;
```

```
    DBMS_OUTPUT.PUT_LINE('Name : ' || sname);
```

```
    DBMS_OUTPUT.PUT_LINE('Marks : ' || smarks);
```

```
    -- NESTED IF
```

```
    IF smarks >= 35 THEN
```

```
        IF smarks >= 60 THEN
```

```
            DBMS_OUTPUT.PUT_LINE('Result: First Class');
```

```
        ELSE
```

```

        DBMS_OUTPUT.PUT_LINE('Result: Pass');
    END IF;
ELSE
    DBMS_OUTPUT.PUT_LINE('Result: Fail');
END IF;
-- CASE statement
CASE
    WHEN smarks >= 70 THEN sgrade := 'A';
    WHEN smarks >= 60 THEN sgrade := 'B';
    WHEN smarks >= 50 THEN sgrade := 'C';
    WHEN smarks >= 35 THEN sgrade := 'D';
    ELSE sgrade := 'F';
END CASE;
DBMS_OUTPUT.PUT_LINE('Grade : ' || sgrade);
-- CASE expression
sresult :=
    CASE
        WHEN smarks >= 60 THEN 'First Class'
        WHEN smarks >= 35 THEN 'Pass'
        ELSE 'Fail'
    END;
DBMS_OUTPUT.PUT_LINE('Final Result: ' || sresult);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Student not found');
END;
/

```

Output:

Name : ravi

Marks : 95

Result: First Class

Grade : A

Final Result: First Class

PL/SQL procedure successfully completed.